

# Access and Credential Transfer

---

## Music Podcast Player

### 1 PURPOSE

This document contains instructions for transferring control of the official GitHub repository and the official AWS deployment of the Music Podcast Player between accounts. Since this software is open-source, anyone may deploy this software independently if they choose. For instruction on on deployment, see “Deployment Instructions.”

### 2 GITHUB REPOSITORY

This section discusses how to transfer operational control of the official Music Podcast Player GitHub repository. This section will refer to the original location of the repository as `https://github.com/{old-owner}/{old-name}` and the new location of the repository as `https://github.com/{new-owner}/{new-name}`.

#### 2.1 UPDATE OPENID ROLES IN AWS

The Continuous Deployment actions in the repository rely on OpenID to authenticate with AWS. In order to prevent them from failing, the following AWS roles will need to have their Trust Relationships updated.

1. `github-{environment}-deployment`
2. `github-{environment}-post-deploy`

In each, the `token.actions.githubusercontent.com:sub` field of each should be updated to include `repo:{new-owner}/{new-name}:environment:{environment}`.

The `repo:{old-owner}/{old-name}:environment:{environment}` entry can be kept until the second step is complete in order to allow GitHub actions from the original repository to continue working until the ownership is transferred. This entry should be removed once ownership has been transferred.

#### 2.2 TRANSFER OWNERSHIP

In the original repository, a user with admin permissions or the owner must go to the repository settings, and under General settings, select the button to transfer ownership of the repository. Following this, the new owner must accept the transfer.

Following the transfer, the new owner may review the roles of collaborators under “Collaborators and teams.”

### 3 PODCASTINDEX API CREDENTIALS

The Music Podcast Player depends on the Podcast Index API for its search feature. The Podcast Index requires credentials to be supplied in order to complete an API request. These can be obtained from the Podcast Index API website.

In the event that these are rotated or changed for any reason, they can be updated in an AWS deployment by storing the API key and API secret in the Systems Manager Parameter Store, using the following parameters.

- `/rss-music-player/podcast-index-api/key`
- `/rss-music-player/podcast-index-api/secret`

In a Docker deployment, they can be updated by storing the new credentials in the following environment variables via the `secrets/backend.env` file.

- `RSS_PLAYER_PODCAST_INDEX_KEY`
- `RSS_PLAYER_PODCAST_INDEX_SECRET`

### 4 OFFICIAL DEPLOYMENT

This section describes the process for transferring the official production deployment of the Music Podcast Player to another AWS account. Note that this procedure is designed to be as simple as possible while preserving user data. If downtime is not acceptable, an alternative procedure should be considered. Additionally, this procedure is theoretical and has never been tested.

#### 4.1 DEPLOY ON THE NEW ACCOUNT

Begin by deploying to production on the new account but without Route 53 Hosted Zone or A Record creation. The deployment should have an empty database (aside from the migrations run by the CDK) and a randomized domain through Cloudfront. Email verification will also need to be temporarily disabled, as this account does not have domain access yet.

Currently, we support a flag to disable email verification, but this works by bypassing email verification for development rather than blocking those features, which is what we'd recommend implementing for this step.

Make sure to review "Deployment Instructions" and verify the required Systems Manager Parameter Store secrets are set up. (It's not a bad idea to rotate them either.)

#### 4.2 WRITE LOCK AND DATABASE SNAPSHOT

The old account will then need to enable a write lock mode, which will temporarily make the database read only and write any cached changes. The key is the database needs to be stable and not change until the write lock is lifted.

We have not implemented a write lock or "maintenance mode," though we'd recommend one be implemented in the future and in the event that the deployment needs to be transferred.

Following the write lock taking effect, the old account will need to manually make a snapshot of the database and share access to it with the new account.

### 4.3 RESTORE SNAPSHOT TO NEW DATABASE

In the new account, the snapshot of the database will be restored to the new database. We'd recommend keeping the snapshot for at least a week so the application can be tested and time is given to discover data loss, should it occur for any reason.

### 4.4 POINT DNS TO NEW DEPLOYMENT

The DNS A record in the old account should be updated to point to Cloudfront in the new account. The idea here is that domain transfer can take a significant amount of time and the site could be reopened to users (albeit without email verification features enabled) during that time. That said, this may not work because the site will not be using the certificate for the domain until it has been transferred, and, additionally, without the domain, the CDK will point the frontend toward the Cloudfront URL for the backend, likely causing CORS issues.

### 4.5 TRANSFER DOMAIN AND ENABLE EMAIL VERIFICATION

Finally, transfer the domain from the old account to the new one and update the deployment to use the official domain. Additionally, email verification can be enabled at this point.

### 4.6 NOTES ON THIS PROCEDURE

Since this has not yet been tested, we strongly recommend testing this procedure with a “production” deployment in two development environments before attempting to transfer the official production deployment.